Índice

Guion de presentación: qué mostrar, qué correr y qué explicar **1**
- 0. Antes de presentar (checklist) . . . 1
  - Comandos de verificación (sin gasto de créditos) . . . 1
  - Comandos de demo (revisar antes, no improvisar) . . . 1
  - Comandos para levantar el sistema en vivo . . . 2
  - Comandos que NO se corren en vivo . . . 2
  - Preparación de la demo (30 min antes) . . . 2
- 1. Orden de la presentación . . . 2
- 2. Sección por sección . . . 3
  - Sección 1 — Producto y alcance (Diego) . . . 3
  - Sección 2 — App móvil (Nelson) . . . 3
  - Sección 3 — Publicación en vivo (Daniel) . . . 3
  - Sección 4 — Validación con compañeros (Nelson) . . . 4
  - Sección 5 — Chat (Joaquín) . . . 4
  - Sección 6 — Arquitectura y decisiones (Diego) . . . 5
  - Sección 7 — Transparencia y Responsible AI (Daniel + Nelson) . . . 5
  - Sección 8 — Engineering loops y gestión (Joaquín) . . . 5
  - Sección 9 — Costos (Diego) . . . 5
  - Sección 10 — Lecciones aprendidas (Joaquín, con aportes de todos) . . . 6
- 3. Planes de contingencia . . . 6
- 4. Lo que la presentación NO debe afirmar . . . 6
- 5. Preguntas probables y respuesta corta . . . 7
- 6. Archivos de referencia (abrir durante la presentación) . . . 7

Guion de presentación: qué mostrar, qué correr y qué explicar

Written for: el equipo que presenta, durante la preparación y el día de la demo. Orden y responsables según DEMO-RUNBOOK.md §1. Detalle técnico de cada punto en EXPLICACION-TECNICA.md.

---

0. Antes de presentar (checklist)

Comandos de verificación (sin gasto de créditos)

Correr desde la raíz del repo.

```
pnpm install                        # dependencias
pnpm lint && pnpm typecheck         # calidad: sin errores
pnpm test                           # todas las pruebas (680 según la última corrida)
pnpm -F shared test                 # solo ranking, esquemas y reglas editoriales
pnpm seed:editorial --check         # valida las 18 borradores del corpus editorial (sin tocar la ba
pnpm evals:chat --dry-run           # valida el set dorado del chat sin llamar al modelo
```

Comandos de demo (revisar antes, no improvisar)

Comando	Qué hace	Cuidado
`EVAL_ID_TOKEN=… pnpm demo:prepare --api https://news-api.diegovalenzuela.workers.dev`	Revisa el Worker (`aiMode`, kill switch, saldo) y calienta los digests de las 8 ubicaciones	Necesita el token. Debe terminar con «Todo listo para el ensayo». Gasta centavos

Comando	Qué hace	Cuidado
`pnpm evals:costs --api https://news-api.diegovalenzuela.workers.dev`	Costo real por función y chequeo de la reserva de la demo	Solo lectura
`pnpm report:ai-suggestions --target=prod`	Tabla «IA sugirió vs editor publicó»	Solo lectura. Hoy sale vacía o incompleta: las editoriales se publicaron sin asistente
`pnpm seed:injection --api <url>`	Agrega la noticia maliciosa de prueba al índice	Solo para la prueba de inyección. Quitarla después con `--remove`

Comandos para levantar el sistema en vivo

```
pnpm -F api dev                  # Worker local con wrangler (opcional; hoy la demo usa el Worke
pnpm -F admin dev                # portal en http://localhost:5173
pnpm -F mobile start --tunnel    # Expo; --tunnel en redes de la U
```

Health del Worker desplegado (abrir en navegador o con curl): https://news-api.diegovalenzuela.workers.dev/health. Debe mostrar `aiMode: live` e índice versión 19 según la entrega.

Comandos que NO se corren en vivo

Comando	Por qué no
`pnpm evals:chat` (sin `--dry-run`)	Gasta créditos. Mostrar la corrida ya guardada en `evidence/evals/`
`pnpm -F api run deploy`	Despliegue. Hacerlo antes, nunca en la presentación
`pnpm seed` / `pnpm seed:editorial` sin `--check`	Escriben en la base. Usar solo si el plan lo pide

Preparación de la demo (30 min antes)

- ☐ Índice reconstruido **desde el portal** (`/news` → «Reconstruir índice»). No usar `pnpm seed` ni «Indexar corpus semilla»: dejan fuera las editoriales.
- ☐ `demo:prepare` terminó con «Todo listo para el ensayo».
- ☐ Ventana del feed: si la última publicación tiene más de 72 h, ajustar `config/public.feedWindowHours` y anotarlo en F4-09.
- ☐ Feed visible en `gt-guatemala`, `mx-cdmx` y `es-madrid`.
- ☐ Flags: `chatMode=full`, `imageGenEnabled=false`.
- ☐ Noticia maliciosa de F4-07 **fuera** del índice.
- ☐ Dos borradores listos con fuentes ya cargadas: D1 (lluvias Quetzaltenango, local, esencial, `en_desarrollo`) y D2 (BCE mantiene tasas, internacional, `confirmada`, sin imagen).
- ☐ `n-ar-003` (dinosaurio argentino, `confirmada`) publicada e indexada, lista para retractar. Comprobar que la búsqueda la devuelve primera.
- ☐ Hotspot propio listo. Plan B nativo (iPhones) y APK (Android) instalados.
- ☐ Videos de respaldo por sección en `evidence/demo-videos/` (hoy no existen: pendiente).

---

1. Orden de la presentación

Responsables según el runbook. La demo funcional es el centro; el orden es obligatorio.

#	Sección	Presenta	Duración sugerida
1	Producto y alcance	Diego	3 min
2	App móvil	Nelson	5 min

#	Sección	Presenta	Duración sugerida
3	Publicación en vivo	Daniel	5 min
4	Validación con compañeros	Nelson	3 min
5	Chat	Joaquín	6 min
6	Arquitectura y decisiones	Diego	4 min
7	Transparencia y Responsible AI	Daniel + Nelson	4 min
8	Engineering loops y gestión	Joaquín	3 min
9	Costos	Diego	3 min
10	Lecciones aprendidas	Joaquín	2 min

Duraciones: estimación propia, no están en el runbook. Ajustar en el ensayo.

---

2. Sección por sección

Sección 1 — Producto y alcance (Diego)

Mostrar: - Problema: noticias generales no sirven a quien vive en otra ciudad; y la IA en noticias sin verificación puede dar confianza falsa. - Dos usuarios: lector (app) y editor (portal). - Checklist del enunciado en PLAN.md §1, con lo cumplido y lo parcial.

Explicar: - Qué está cumplido: ubicación simulada sin GPS, chat inicial, fuentes y certeza heredadas, publicación con procedencia, control de costos. - Qué está parcial: login con Google en la app y el portal (capturas fuera del repo), feed con jerarquía visual, prueba con compañeros. - Ver ENTREGA.md §2. Decir lo parcial antes de que lo pregunten.

Evidencia: PLAN.md §1, ENTREGA.md.

Sección 2 — App móvil (Nelson)

Mostrar en vivo (iPhone y, si se puede, Android): 1. Login con Google (puente web). 2. Chat como pantalla inicial. 3. Feed: esenciales arriba, tarjetas con tiers distintos. 4. Abrir una noticia: pie «Respuesta generada con IA» no aplica aquí; mostrar «Cómo se hizo esta noticia» (fuentes, certeza, quién publicó). 5. Cambiar ubicación y mostrar que el feed cambia. 6. «¿Por qué veo esto?» en una tarjeta.

Explicar: - Expo Go: por qué no hay módulos nativos y qué es el puente web (ADR-001, ADR-003). - El feed se ordena en el teléfono con `rankFeed` de `packages/shared`. Sin LLM. - Solo probado en iPhone 16 Pro Max. Android sin probar el chat.

Evidencia: F3-04, F3-06, F4-04, F3-09.

Archivos para mostrar en el código si preguntan: apps/mobile/src/feed/useNewsFeed.ts, apps/mobile/src/services/newsFeed

---

Sección 3 — Publicación en vivo (Daniel)

Mostrar en el portal (`/news`): 1. Abrir D1 (lluvias, `en_desarrollo`). Mostrar fuentes, nota de certeza y checklist. 2. Intentar publicar sin cumplir una regla, por ejemplo `confirmada` con una sola fuente. Mostrar que el botón se bloquea y el mensaje. 3. Ajustar a `en_desarrollo` con nota y publicar. 4. Publicar D2 (BCE) con portada tipográfica. Mostrar la etiqueta «Portada generada · no es foto». 5. Abrir «Sugerir con IA» en un borrador y mostrar que las sugerencias son eso: sugerencias, con el editor decidiendo.

Explicar: - Por qué la IA no puede publicar ni asignar certeza (ADR-006). - Qué pasa al publicar: `POST /admin/index/upsert`, embedding, índice en KV. - Por qué las reglas están en `packages/shared` y no solo en la interfaz.

Evidencia: validatePublish.ts, F2-07-image-rights-fix.md, F2-11-corpus-prod.md.

Sección 4 — Validación con compañeros (Nelson)

Mostrar: 1. Compañeros con ubicaciones distintas miran el feed de `gt-quetzaltenango`, `gt-guatemala` y `es-madrid`. 2. D1 debe aparecer en «Lo que debes saber» para las dos ubicaciones de Guatemala, con tier distinto, y **no** en el bloque esencial de Madrid. 3. Proyectar `/compare` con D1 y D2 y las 8 ubicaciones. Mostrar posición y tier.

Explicar: - El comparador usa el mismo `rankFeed` que la app: lo que se ve en el portal es lo que ve el lector. - Cómo se evita la burbuja: cuotas y diversidad (LOOP-009).

Evidencia: F3-11-comparator.md, captures/F3-11-compare-diversidad.jpg.

Advertencia: la validación con compañeros no se ensayó. Si el Wi-Fi falla, usar los teléfonos del equipo con ubicaciones distintas.

Sección 5 — Chat (Joaquín)

Preguntas en este orden, desde el teléfono proyectado. Cada respuesta se abre en «¿Cómo se generó?».

#	Ubicación	Pregunta	Qué debe verse
1	`gt-guatemala`	«Resume lo más importante de hoy»	Modo digest; costo 0 desde la segunda consulta
2	`mx-cdmx`	«¿Qué se sabe de las lluvias en Guatemala?»	Chip «En desarrollo» heredado de D1; fuente tocable
3	`gt-guatemala`	«Explícame lo más importante de México»	Intención «otro país»; cita de noticia
4	`us-nueva-york`	«¿Qué se acordó en la cumbre climática?»	Tema; fuente que abre la lectura
5	`es-madrid`	«¿Quién ganó el mundial de ajedrez de 1972?»	Abstención; pie «Sin IA»; costo 0
6	`gt-guatemala`	«¿Qué se sabe del dinosaurio que describieron científicos argentinos?»	Antes de retractar: cita `n-ar-003`. Después de retractar (Daniel): ya no la cita

Explicar: - Pipeline en 6 pasos: intención → caché → embedding → coseno → abstención o LLM → validación de citas. - Por qué la abstención no cuesta: sin resultados sobre τ, no se llama al modelo. - Por qué la certeza viene de la base de datos y no del modelo. - Si el proveedor falla: `retrieval_only`, que muestra fuentes sin texto generado.

Números para decir (con la fuente abierta): - Set dorado v4, 34 casos: abstención 5/5, 0 citas prohibidas, 0 fallos ante inyección. - Latencia de una corrida: respuesta p50 2.8 s, p95 4.1 s; abstención 0.6 s; caché 0.3 s. Decir «una sola corrida». - Decir también: τ = 0.45 no está calibrado.

Ejecutar si preguntan: `pnpm evals:chat --dry-run` (valida el set sin gastar). No correr el set completo en vivo.

Evidencia: evidence/evals/chat-20261013-0440.json, evidence/evals/chat-injection-20261013-0419.json, F4-01, F4-02, F4-05.

Archivos para mostrar en el código: services/api/src/rag/chat.ts (función `runChat`), services/api/src/rag/search.ts (coseno).

Sección 6 — Arquitectura y decisiones (Diego)

Mostrar: 1. Diagrama de ARQUITECTURA.md §1. Recorrerlo con el dedo: app → Firestore; portal → Worker; Worker → KV, D1, Workers AI, Anthropic. 2. Regla «IA al publicar, código al leer» (ARQUITECTURA.md §3). 3. Tres ADRs clave: 002 (ranking sin LLM), 006 (certeza humana), 012 (modelo Haiku 5.5). Mostrar DECISIONES.md. 4. Latencias del chat (ver sección 5).

Explicar: - Por qué Cloudflare Workers y no Firebase Functions: plan gratuito y sin Admin SDK (ADR-004). - Por qué búsqueda por fuerza bruta: el corpus es pequeño (ADR-005). - Frontera del gateway: un solo punto de salida a proveedores, con prueba automatizada (boundary.test.ts).

Ojo: ADR-012 reemplaza a ADR-009, pero no está escrito por qué se cambió de 4.5 a 5.5. Decirlo así si preguntan.

Evidencia: DECISIONES.md, MODELO-DATOS.md, CONTRATOS-API.md.

Sección 7 — Transparencia y Responsible AI (Daniel + Nelson)

Daniel (fuentes, validación, imágenes): 1. Abrir «Cómo se hizo esta noticia» de una noticia `confirmada`. Mostrar fuentes con fecha de consulta. 2. Mostrar la etiqueta de cada salida de IA (VERIFICACION-Y-FUENTES.md §4). 3. **Retractación en vivo:** Daniel retracta `n-ar-003` desde el portal con corrección. Joaquín repite la pregunta del chat y ya no se cita. Prueba automatizada: demoFlow.test.ts.

Nelson (importancia, burbuja): 1. Abrir «¿Por qué veo esto?» y mostrar los componentes de un puntaje. 2. Mostrar que una noticia esencial aparece aunque el usuario no tenga interés en ese tema.

Explicar: - Matriz RAI: cada pregunta del enunciado → decisión → pantalla → prueba (RAI-MATRIZ.md). - Lo que está □ en la matriz sigue en □: falta la captura en teléfono. - La IA sugiere, el editor decide. Medición previa: importancia distinta en 5 de 5 casos (LOOP-005).

Evidencia: F3-03-explanations.md, F3-05-images.md, F3-12-manual-review.md.

Sección 8 — Engineering loops y gestión (Joaquín)

Tres loops con antes y después, uno por diapositiva. Del PRESENTACION-EVIDENCIA.md:

1. **LOOP-006 — Una búsqueda web no basta.** Antes: redactar 16 noticias con resúmenes de búsqueda. Después: 4 resúmenes eran falsos o desactualizados; ningún dato entra sin leer la fuente original. Archivo.
2. **LOOP-009 — Cuotas y diversidad.** Antes: algoritmo voraz. 2 fallas en 182 pruebas. Después: selección que reserva separadores y cuotas. Archivo.
3. **LOOP-016 — El chat ignoraba la ubicación.** Antes: «No tengo forma de saber en qué ciudad vives». Después: precisión de citas 85.7 % → 96.0 %. Archivo.

Alternativas si hay tiempo: LOOP-015 (falsas abstenciones del digest) y LOOP-005 (costo y calidad de la IA del portal).

Mostrar: la carpeta evidence/loops/ con los 18 registros. Mencionar el DoD en ENGINEERING-LOOPS.md y el tablero en PLAN.md §6.

Ojo: algunos loops se hicieron con otro agente (Codex), no con Claude Code. Decirlo si preguntan.

Sección 9 — Costos (Diego)

Mostrar en vivo: portal `/costs` (dashboard de costos). Costo por función, llamadas evitadas y reserva restante.

Ejecutar (solo lectura):

```
pnpm evals:costs --api https://news-api.diegovalenzuela.workers.dev
```

Números para decir: - Gasto real según el ledger: USD 0.0605 de USD 20. Consola de Anthropic: USD 0.06. Diferencia menor a 1 %. - Costo por función: `chat_answer` USD 0.000296; `digest` USD 0.000809; `enrich` USD 0.003183 (con Haiku 4.5; con 5.5 sin medir). - Ahorros: 21 % de 357 llamadas resueltas sin LLM (abstención, caché, digest compartido). - Reserva de USD 7 para la demo, intacta. Estimación de la demo USD 0.075 → cobertura 92.8×. - Kill switch y modo degradado probados.

Explicar: - Cómo el gateway registra cada llamada con tokens reales (gateway.ts). - Por qué el saldo se calcula como 20 − 0.06 = 19.94 y no se leyó exacto en la consola. Decirlo así.

Evidencia: PRESUPUESTO-IA.md, F4-08-costs.md, LOOP-003.

Sección 10 — Lecciones aprendidas (Joaquín, con aportes de todos)

Borrador en LECCIONES-APRENDIDAS.md. Tres ideas que se pueden decir con evidencia:

1. Una búsqueda no es fuente: se verifica contra el original (LOOP-006).
2. Un modelo que no ve el contexto del lector responde mal con seguridad: la ubicación tuvo que entrar al prompt (LOOP-016).
3. La IA sugiere; medirla contra la decisión humana mostró que no basta (LOOP-005).

Límites para decir en voz alta: - No hay fecha del hecho (LOOP-017). - No hay detección de duplicados (LOOP-018). - Muestras pequeñas en evals. - Sin pruebas completas en teléfono.

Qué haríamos distinto: lo que el equipo escriba aquí. El documento de lecciones aún es borrador.

---

3. Planes de contingencia

Falla	Respuesta
Wi-Fi del aula no deja entrar a compañeros	Hotspot propio; si falla, teléfonos del equipo con ubicaciones distintas
Expo Go no carga	Plan B nativo en iPhone; APK en Android
Proveedor LLM caído o lento	`chatMode = retrieval_only` desde `/costs`; explicarlo como degradación controlada
Saldo inesperadamente bajo	Mismo `retrieval_only`; mostrar el dashboard como prueba de control
Todo falla	Videos de respaldo por sección (hoy no existen en `evidence/demo-videos/`)

---

4. Lo que la presentación NO debe afirmar

Según ENTREGA.md §3–4:

- Que el feed fue medido en teléfono (ranking < 20 ms, cambio de ubicación < 300 ms, dos teléfonos < 5 s): pendiente.
- Que el chat fue probado en Android: pendiente.
- Que el corpus tiene 55 noticias: tiene 53.
- Que hubo ensayos generales de la demo: 0 de 3.
- Que los 3 videos de respaldo existen: no existen.
- Que la latencia es estable: es una sola corrida.
- Que τ está calibrado: no está.
- Que la tabla «IA sugirió vs editor publicó» tiene datos de producción: no tiene.

- Que se detectan duplicados o fechas del hecho: no.

---

5. Preguntas probables y respuesta corta

Pregunta	Respuesta corta	Dónde mostrar
¿Por qué no usan un LLM para ordenar el feed?	Costo cero, latencia, y se puede explicar cada punto	`score.ts`, ADR-002
¿Qué impide que el modelo invente una fuente?	Las citas se validan contra lo recuperado en código; si no coinciden, abstención	`chat.ts` paso 9
¿Qué pasa si el modelo dice que algo está confirmado?	Nada: la certeza viene de la base de datos	`chat.ts` paso 10
¿Por qué Haiku 5.5?	Decisión de ADR-012. Motivo no documentado: decirlo	ADR-012
¿Cuánto cuesta una pregunta?	Entre 0 y USD 0.0003, según el camino	`/costs`, `evals:costs`
¿Cómo sabe que no lo engañaron con una inyección?	Set de 3 casos, 0 fallos. Muestra pequeña	`evidence/evals/chat-injection-*.json`
¿Funciona en Android?	Feed y login sí en pruebas automáticas; el chat no está probado en Android	CAPTURAS-PENDIENTES.md
¿Por qué la retractación no borra la noticia?	Se conserva con su corrección; es una regla de VERIFICACION-Y-FUENTES.md §6	Mostrar `n-ar-003` retractada
¿Qué pasa con una noticia vieja publicada hoy?	Parece reciente. Limitación declarada	LOOP-017
¿Quién decide la importancia?	El editor, con sugerencia de IA que se audita	LOOP-005
¿Cómo se evita la burbuja?	Esenciales sin personalizar, cuotas nacional e internacional, diversidad de temas	`antibubble.ts`
¿Qué pasa si se acaba el dinero?	Kill switch y `retrieval_only`; la reserva de USD 7 cubre la demo	`/costs`

---

6. Archivos de referencia (abrir durante la presentación)

Qué	Archivo
Fórmula del feed	packages/shared/src/ranking/score.ts
Orden completo del feed	packages/shared/src/ranking/rankFeed.ts
Anti-burbuja	packages/shared/src/ranking/antibubble.ts
Reglas de certeza	packages/shared/src/editorial/validatePublish.ts
Pipeline del chat	services/api/src/rag/chat.ts
Gateway y kill switch	services/api/src/ai/gateway.ts
Prompt del chat	services/api/src/ai/prompts/chat.v2.ts
Frontera de IA (prueba)	services/api/src/ai/boundary.test.ts
Flujo de publicación del portal	apps/admin/src/lib/publishFlow.ts
Feed en la app	apps/mobile/src/feed/useNewsFeed.ts
Guion del runbook	DEMO-RUNBOOK.md